



Recitation 6

Recitation Overview

- Implementing functions that work with binary numbers in C

Instructions for tasks

- Copy `/share/recitations/recitation06/bitPositions.c` to your directory.
- We will correct the implementation of function **getLSBpos**, **getMSBpos**, **getNumPos** (functions **getNextBitPos**, **printBP** will be covered after Pessach).
- Copy `/share/recitations/recitation06/test_bitPositions.c` to your directory. This file contains a single main function that runs tests for all functions.
- To generate a program that runs the execution examples for task X, compile as follows:

```
gcc -Wall bitPositions.c test_bitPositions.c -D TEST_X -o test_X
```
- The `-D TEST_X` 'turns on' the code that is wrapped with the `#ifdef TEST_X` directives.
- This triggers an 'unused variable' in the function of Task 4, but you can ignore this warning. The executable `test_X` is generated and you can run it and compare the results with what we report above.

Definition of $bp()$

- For a given **int** a , we specify its binary representation by $(a_k, \dots, a_1, a_0)$. We denote by $bp(a)$ the bit positions that are turned on in the representation of a : $bp(a) = \{i | a_i = 1\}$.
For instance, the binary representation of $a = 4$ is $(1,0,0)$, and for $a = 13$ it is $(1,1,0,1)$.
The bit positions that are turned on in the binary representations are as follows:

$$bp(4) = \{2\} \text{ and } bp(13) = \{0,2,3\}.$$

Task 1

- Complete the implementation of the function **getLSBpos**, such that it returns the smallest position in which the binary representation of parameter a is turned on ($\min\{i | i \in bp(a)\}$) (LSB stands for *least significant bit*). If $a = 0$ and $bp(a) = \emptyset$, the function should return -1. You should modify the code only in the three marked places and **use a recursive call in the third expression**.
- **getLSBpos(4)** returns 2
- **getLSBpos(24)** returns 3
- **getLSBpos(0)** returns -1
- **getLSBpos(0xA00B)** returns 0
- **getLSBpos(0xA6C0)** returns 6
- **getLSBpos(-1)** returns 0
- **getLSBpos(-24)** returns 3

$$\begin{aligned}
 4_{10} &= 100_2 \\
 24_{10} &= 11000_2
 \end{aligned}$$

But first – a reminder of number representations

2^x	7	6	5	4	3	2	1	0
position value	128	64	32	16	8	4	2	1
bit/digit value	0	0	0	1	1	0	0	0

$$24 = 0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$$

But first – a reminder of number representations

16^x	3	2	1	0
position value	4096	256	16	1
digit value	A (10)	0	0	B (11)

$$\begin{aligned}
 0xA00B &= A \cdot 16^3 + 0 \cdot 16^2 + 0 \cdot 16^1 + B \cdot 16^0 = \\
 &= 10 \cdot 16^3 + 0 \cdot 16^2 + 0 \cdot 16^1 + 11 \cdot 16^0 = \\
 &= 40971
 \end{aligned}$$

But first – a reminder of number representations

2^x	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
position value	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
bit/digit value	1	0	1	0	0	0	0	0	0	0	0	0	1	0	1	1

But first – a reminder of number representations

2^x	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
position value	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
bit/digit value	1	0	1	0	0	0	0	0	0	0	0	0	1	0	1	1
Hexa-decimal digit	A				0				0				B			
Decimal value	10				0				0				11			

Two's complement for negative numbers

We require that $x + (-x)$ will result in 0 in regular binary vector addition:

$$\begin{array}{r}
 1 \\
 + \\
 -1
 \end{array}
 \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|}
 \hline
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\
 \hline
 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
 \hline
 \hline
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 \hline
 \end{array}$$

1

Overflow

Two's complement for negative numbers

How to get the negative representation of a certain number using the two's complement method

1

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Step 1: Invert all bits of the given number

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Step 2: add +1 to the least significant bit (LSB)

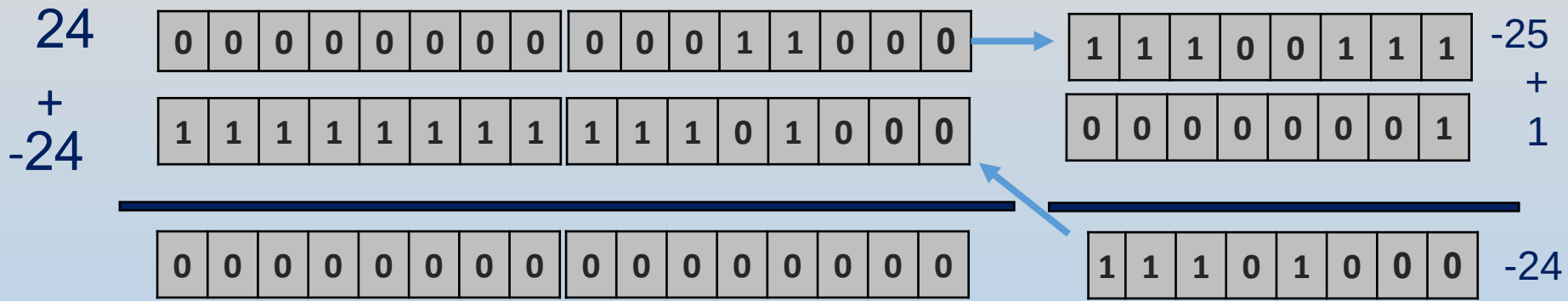
-1

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

From negative to positive follow the same steps

Two's complement for negative numbers

We require that $x + (-x)$ will result in 0 in regular binary vector addition:



- A vector's complement is obtained by flipping all bits (0→1, 1→0)
- A vector + its complement equals -1

➔ $-x = (\text{complement of } x) + 1$

Task 1

- Complete the implementation of the function **getLSBpos**, such that it returns the smallest position in which the binary representation of parameter a is turned on ($\min\{i | i \in bp(a)\}$) (LSB stands for *least significant bit*). If $a = 0$ and $bp(a) = \emptyset$, the function should return -1. You should modify the code only in the three marked places and **use a recursive call in the third expression**.
- **getLSBpos(4)** returns 2
- **getLSBpos(24)** returns 3
- **getLSBpos(0)** returns -1
- **getLSBpos(0xA00B)** returns 0
- **getLSBpos(0xA6C0)** returns 6
- **getLSBpos(-1)** returns 0
- **getLSBpos(-24)** returns 3

$$4_{10} = 100_2$$

$$24_{10} = 11000_2$$

Task 1 - Answer

```
int getLSBpos(int a) {  
    if (a == 0) {  
        return -1;  
    }  
    if (a & 1 == 1) {  
        return 0;  
    }  
    return getLSBpos(a >> 1) + 1;  
}
```

Task 2

- Complete the implementation of the function **getMSBpos**, such that it returns the largest position in which the binary representation of parameter a is turned on ($\max\{i \mid i \in bp(a)\}$) (MSB stands for *most significant bit*). If $a = 0$ and $bp(a) = \emptyset$, the function should return -1. You should modify the code only in the two marked places and **use a recursive call in the second expression**. Try to use bitwise and logical operators when appropriate.
- **Note:** negative numbers may require some special consideration here. If your solution requires knowledge of the number of bytes that **int** takes, then you may obtain this number using the **sizeof()** operator. However, try to find a solution that does not require this knowledge.

$$\begin{aligned} 4_{10} &= \mathbf{1}00_2 \\ 24_{10} &= \mathbf{1}1000_2 \end{aligned}$$

Task 2 - Answer

```
int getMSBpos(int a) {  
    if (a == 0) {  
        return -1;  
    }  
  
    return 1 + getMSBpos(((unsigned)a) >> 1);  
}
```


Task 3

- Complete the implementation of the function **getNumPos**, such that it returns the number of positions in which the binary representation of parameter a are turned on ($|\{i | i \in bp(a)\}|$). You should modify the code only in the two marked places and **use a recursive call in the second expression**. Try to use bitwise and logical operators when appropriate.
- **Note:** the same issue with negative numbers is also relevant here.
- **Execution examples:** (same inputs we used in Task 1)
- **getNumPos(4)** returns 1
- **getNumPos(24)** returns 2
- **getNumPos(0)** returns 0
- **getNumPos(0xA00B)** returns 5
- **getNumPos(0xA6C0)** returns 6
- **getNumPos(-1)** returns 32 (depends on the size of **int**)
- **getNumPos(-24)** returns 28 (depends on the size of **int**)

$$4_{10} = \mathbf{1}00_2$$
$$24_{10} = \mathbf{11}000_2$$

Task 3 - Answer

```
int getNumPos(int a) {  
    if (a == 0) {  
        return 0;  
    }  
  
    return getNumPos(((unsigned)a) >> 1) + (a & 1);  
}
```